



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 5:

BIT PERMUTATION-PART II



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Octubre 18, 2025

EJERCICIOS DE PROGRAMACIÓN I

- b) Implementa una **función** que reciba como parámetro un entero $n > 0$, donde n es la longitud de M . La función debe **generar n caracteres aleatorios**, estos serán K , y debe **devolver K** .

```
1 // Función para generar una llave aleatoria
2 void genLlave(unsigned char* clave, int longitud) {
3     for (int i = 0; i < longitud; i++) {
4         clave[i] = rand() % 256;
5     }
6 }
```

- c) Implementa una **función** que reciba como parámetros el texto plano M y la clave pseudoaleatoria K , y que cifre M usando el **operador bit a bit XOR ($\oplus$)** como se describió arriba. La función debe almacenar C en un arreglo de caracteres.

```
1 // Función para cifrar/descifrar usando XOR
2 void xorCipher(const char* entrada, const unsigned char* clave, char* salida, int longitud) {
3     for (int i = 0; i < longitud; i++) {
4         salida[i] = entrada[i] ^ clave[i];
5     }
6     salida[longitud] = '\0';
7 }
```

- d) Implementa una **función** para **imprimir en hexadecimal** los valores de M , K y C .

```
1 // Función para imprimir los valores
2 void printValores(char* cifrado, unsigned char* clave, char* mensaje, int longitud) {
3     printf("\nMensaje en claro: ");
4     for (int i = 0; i < longitud; i++) {
5         printf("%02X ", (unsigned char)mensaje[i]);
6     }
7     printf("- %s", mensaje);
8
9     printf("\nLlave utilizada: ");
10    for (int i = 0; i < longitud; i++) {
11        printf("%02X ", clave[i]);
12    }
13
14    printf("\nMensaje cifrado: ");
15    for (int i = 0; i < longitud; i++) {
16        printf("%02X ", (unsigned char)cifrado[i]);
17    }
18    printf("- %s", cifrado);
19 }
```

Sesión de Laboratorio 5: Bit Permutation-Part II

e) ¿Cómo puedes descifrar un texto cifrado dado C y la clave K?

Volviendo a aplicar un XOR al texto cifrado con la misma clave con la que cifro.

Para demostrar esto le vamos a enviar a nuestra función de cifrado el mensaje cifrado como arreglo de entrada y la clave con la que se cifro, y como resultado debe de dar el texto en claro.

```
1  int main (){
2      srand(time(NULL));
3
4      char mensaje[100], cifrado[100];
5      unsigned char clave[100];
6
7      printf("Ingrese el mensaje a cifrar: ");
8      fgets(mensaje, sizeof(mensaje), stdin);
9
10     int longitud = strlen(mensaje) - 1;
11
12     genLlave(clave, longitud);
13     xorCipher(mensaje, clave, cifrado, longitud);
14     xorCipher(cifrado, clave, mensaje, longitud);
15     printValores(cifrado, clave, mensaje, longitud);
16
17     return 0;
18 }
```

Ejecución:

Ingrese el mensaje a cifrar: Hoy es un dia soleado

Mensaje en claro: 48 6F 79 20 65 73 20 75 6E 20 64 69 61 20 73 6F 6C 65 61 64 6F - Hoy es un dia soleado

Llave utilizada: 3F 1C FA ED A3 43 CA F3 6B 99 FF EC FA C1 44 27 6E 34 BC 87 30

Mensaje cifrado: 77 73 83 CD C6 30 EA 86 05 B9 9B 85 9B E1 37 48 02 51 DD E3 5F - wsâ=ã0Ûâ||øàøß7HQ|ò_

Mensaje descifrado: 48 6F 79 20 65 73 20 75 6E 20 64 69 61 20 73 6F 6C 65 61 64 6F - Hoy es un dia soleado

OPERACIÓN DE SUSTITUCIÓN

a) Implementa una función que genere aleatoriamente una función

$S: 0,1^4 \rightarrow 0,1^4$. Tu función debe **almacenar la tabla generada de S en un archivo de texto**.

```
1 // Función para generar la función S y guardarla en un archivo
2 void funcionS(const char *archivo, unsigned char s[16]) {
3     for(int i = 0; i < 16; i++) {
4         s[i] = i;
5     }
6
7     srand(time(NULL));
8     for(int i = 15; i > 0; i--) {
9         int j = rand() % (i + 1);
10        unsigned char temp = s[i];
11        s[i] = s[j];
12        s[j] = temp;
13    }
14
15    FILE *f = fopen(archivo, "w");
16    if(f == NULL) {
17        printf("Error al abrir archivo\n");
18        return;
19    }
20
21    fprintf(f, "z    | ");
22    for(int i = 0; i < 16; i++) {
23        fprintf(f, "%X ", i);
24    }
25    fprintf(f, "\n");
26
27    fprintf(f, "S(z) | ");
28    for(int i = 0; i < 16; i++) {
29        fprintf(f, "%X ", s[i]);
30    }
31    fprintf(f, "\n");
32    fclose(f);
33 }
```

Sesión de Laboratorio 5: Bit Permutation-Part II

- b) Implementa una función que reciba como entrada un **arreglo de caracteres M** y una **función S**, para sustituir cada bloque de 4 bits en **M** por los cuatro bits dados por **S**. Imprime **M** y el **resultado de la sustitución en hexadecimal**. También imprime **cada paso del proceso de sustitución**. Para implementar esta función, usa **operadores bit a bit**.

```
1 // Función para aplicar la sustitución usando la función S
2 void sustitucion(unsigned char *m, int tam, unsigned char s[16]) {
3     printf("\nEntrada: ");
4     for(int i = 0; i < tam; i++) {
5         printf("%02X ", m[i]);
6     }
7     printf("\n\nPasos:\n");
8
9     for(int i = 0; i < tam; i++) {
10         unsigned char b1 = (m[i] >> 4) & 0x0F;
11         unsigned char b2 = m[i] & 0x0F;
12
13         unsigned char s1 = s[b1];
14         unsigned char s2 = s[b2];
15
16         printf("\nBloque %d (%02X):\n", i, m[i]);
17         printf("  S(%X) = %X\n", b1, s1);
18         printf("  S(%X) = %X\n", b2, s2);
19
20         m[i] = (s1 << 4) | s2;
21         printf("  Resultado: %02X\n", m[i]);
22     }
23
24     printf("\nResultado: ");
25     for(int i = 0; i < tam; i++) {
26         printf("%02X ", m[i]);
27     }
28     printf("\n");
29 }
```

Sesión de Laboratorio 5: Bit Permutation-Part II

Ejecución:

```
Introduce texto: Hola
Funcion S generada y guardada en 's.txt'

Entrada: 48 6F 6C 61

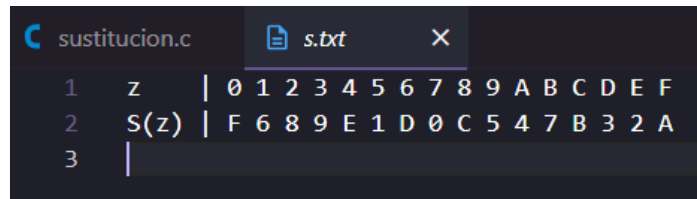
Pasos:
Bloque 0 (48):
  S(4) = E
  S(8) = C
  Resultado: EC

Bloque 1 (6F):
  S(6) = D
  S(F) = A
  Resultado: DA

Bloque 2 (6C):
  S(6) = D
  S(C) = B
  Resultado: DB

Bloque 3 (61):
  S(6) = D
  S(1) = 6
  Resultado: D6

Resultado: EC DA DB D6
```



The screenshot shows a terminal window with two tabs: 'sustitucion.c' and 's.txt'. The 's.txt' tab is active and displays a substitution table. The table has three rows and 26 columns. The first row is labeled 'z' and the second row is labeled 'S(z)'. The third row is empty. The columns are labeled with hexadecimal values 0 through F, and then A through F.

1	z		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	S(z)		F	6	8	9	E	1	D	0	C	5	4	7	B	3	2	A
3																		